

```
import sagemaker
from sagemaker import get_execution_role
import boto3, os, numpy as np
from sklearn.linear_model import LinearRegression
import joblib

# SageMaker setup
region = boto3.Session().region_name
session = sagemaker.Session()
role = get_execution_role()

print("✅ Region:", region)
print("✅ Role:", role)
print("✅ Default bucket:", session.default_bucket())
```

```
sagemaker.config INFO - Not applying SDK defaults from location: /etc/xdg/sagemaker/config.yaml
sagemaker.config INFO - Not applying SDK defaults from location: /home/ec2-user/.config/sagemaker/config.yaml
✅ Region: us-east-1
✅ Role: arn:aws:iam::529166310577:role/service-role/AmazonSageMaker-ExecutionRole-20250930T092648
✅ Default bucket: sagemaker-us-east-1-529166310577
```

```
# Generate dummy data
X = np.random.rand(100, 1)
y = 2 * X.squeeze() + np.random.randn(100) * 0.1

# Train model
model = LinearRegression()
model.fit(X, y)

# Save model
os.makedirs("model", exist_ok=True)
joblib.dump(model, "model/model.joblib")

print("✅ Model trained and saved locally!")
```

```
✅ Model trained and saved locally!
```

```
# Compress model for SageMaker
import tarfile

with tarfile.open("model.tar.gz", "w:gz") as tar:
    tar.add("model", arcname=".")

# Upload
model_path = session.upload_data("model.tar.gz", key_prefix="inference-test-model")
print("✅ Model uploaded to S3:", model_path)
```

```
✅ Model uploaded to S3: s3://sagemaker-us-east-1-529166310577/inference-test-model/model.tar.gz
```

```
from sagemaker import image_uris
from sagemaker.model import Model

# Get sklearn container image
image_uri = image_uris.retrieve(
    framework="sklearn",
    region=region,
    version="1.2-1"
)

# Create model resource
model = Model(
    image_uri=image_uri,
    model_data=model_path,
    role=role,
    name="scratch-sklearn-model",
    sagemaker_session=session
)

print("✅ SageMaker model object created!")
```

```
✅ SageMaker model object created!
```

```
import boto3
sm_client = boto3.client("sagemaker")
```

```
# List all endpoints
endpoints = sm_client.list_endpoints()
for ep in endpoints["Endpoints"]:
    print(ep["EndpointName"], "-", ep["EndpointStatus"])
```

```
scratch-inference-endpoint - Creating
xgb-fixed-1762756784-endpoint - InService
xgboost-loadtest-1762754915-endpoint - Failed
```

```
import boto3, time
sm_client = boto3.client("sagemaker")

while True:
    resp = sm_client.describe_endpoint(EndpointName="scratch-inference-endpoint")
    status = resp["EndpointStatus"]
    print("Current status:", status)
    if status == "InService":
        print("✅ Endpoint is live and ready for inference!")
        break
    elif status == "Failed":
        print("❌ Endpoint creation failed.")
        print("Reason:", resp.get("FailureReason", "No reason reported."))
        break
    time.sleep(20)
```

```
Current status: Creating
Current status: Creating
Current status: Creating
Current status: Creating
Current status: Creating
Current status: Creating
Current status: Failed
❌ Endpoint creation failed.
Reason: The primary container for production variant AllTraffic did not pass the ping health check. Please check CloudWatch
```

```
import joblib, os, tarfile
from sklearn.linear_model import LinearRegression
import numpy as np

# retrain simple model
X = np.random.rand(100, 1)
y = 2 * X.squeeze() + np.random.randn(100) * 0.1
model = LinearRegression()
model.fit(X, y)

# save as model.pkl (required by container)
os.makedirs("model", exist_ok=True)
joblib.dump(model, "model/model.pkl")

# tar it
with tarfile.open("model.tar.gz", "w:gz") as tar:
    tar.add("model", arcname=".")
print("✅ Repacked model as model.pkl")
```

```
✅ Repacked model as model.pkl
```

```
model_path = session.upload_data("model.tar.gz", key_prefix="inference-fixed-model")
print("Model uploaded to:", model_path)
```

```
Model uploaded to: s3://sagemaker-us-east-1-529166310577/inference-fixed-model/model.tar.gz
```

```
import boto3
sm_client = boto3.client("sagemaker")

resp = sm_client.describe_endpoint(EndpointName="scratch-inference-endpoint-fixed")
print(resp["EndpointStatus"])
```

```
import joblib, os, tarfile
from sklearn.linear_model import LinearRegression
import numpy as np

# Simple retrain (or load your existing model)
X = np.random.rand(100, 1)
y = 2 * X.squeeze() + np.random.randn(100) * 0.1
model = LinearRegression()
```

```
model.fit(X, y)

# Save directly as model.pkl (not inside subfolder)
joblib.dump(model, "model.pkl")

# Tar correctly – no nested folders!
with tarfile.open("model.tar.gz", "w:gz") as tar:
    tar.add("model.pkl", arcname="model.pkl")

print("✅ Correctly packaged model.tar.gz created.")
```

✅ Correctly packaged model.tar.gz created.

```
model_path = session.upload_data("model.tar.gz", key_prefix="fixed-model-upload")
print("✅ Uploaded model to:", model_path)
```

✅ Uploaded model to: s3://sagemaker-us-east-1-529166310577/fixed-model-upload/model.tar.gz

```
import boto3

runtime = boto3.client('sagemaker-runtime')

# Input must be a CSV string (no brackets, no JSON)
payload = "0.6"

response = runtime.invoke_endpoint(
    EndpointName='xgb-fixed-1762756784-endpoint',
    ContentType='text/csv', # ✅ use CSV content type
    Body=payload
)

result = response['Body'].read().decode('utf-8')
print("🟡 XGBoost prediction:", result)
```

🟡 XGBoost prediction: 0.5661734938621521

```
payload = "0.1\n0.5\n0.9"

response = runtime.invoke_endpoint(
    EndpointName='xgb-fixed-1762756784-endpoint',
    ContentType='text/csv',
    Body=payload
)

result = response['Body'].read().decode('utf-8')
print("🟡 Predictions:\n", result)
```

🟡 Predictions:
0.14679093658924103
0.5064154267311096
0.7143593430519104

```
import pandas as pd

inputs = [0.1, 0.5, 0.9]
predictions = [0.14679093658924103, 0.5064154267311096, 0.7143593430519104]

df = pd.DataFrame({"Input": inputs, "Predicted Output": predictions})
df
```

	Input	Predicted Output
0	0.1	0.146791
1	0.5	0.506415
2	0.9	0.714359

Start coding or [generate](#) with AI.

